

Projeto 1 - Introdução à programação.

25 de agosto de 2023

Universidade de São Paulo - Instituto de Física de São Carlos.

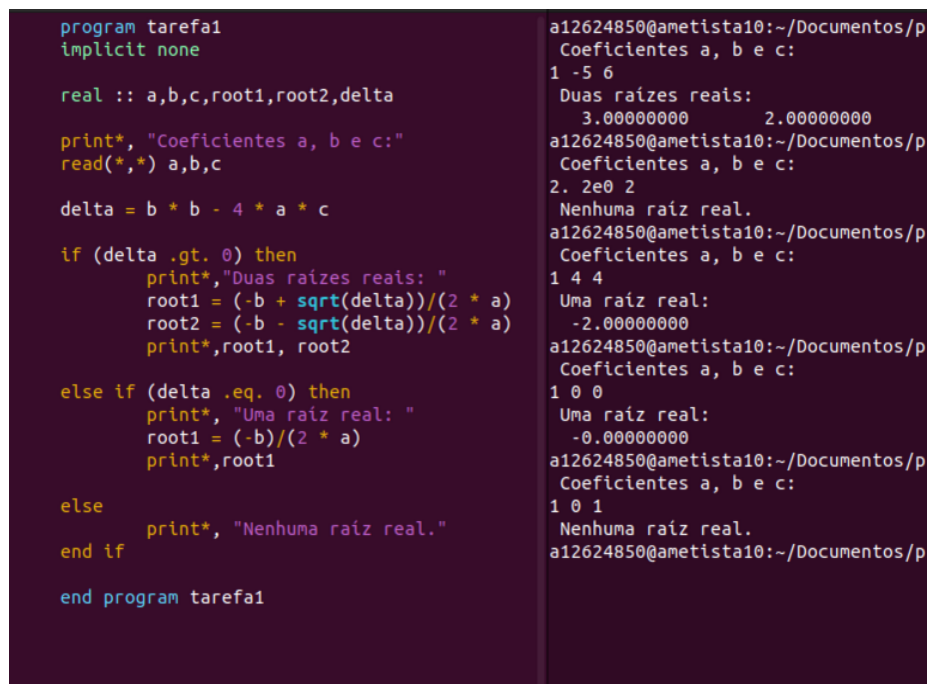
Docente: José A. Hoyos

Tony Maciel Alexander. Número USP: 12624850

1 Tarefa 1

Problema: dado os coeficientes de um polinômio de grau dois, determinar quantas e quais raízes reais este tem.

Para determinar o número de raízes que o polinômio tem, foi inicialmente calculado o discriminante: $\Delta = b^2 - 4ac$. Caso este seja negativo, não terá raízes reais e o programa se encerra; caso este for nulo, é impresso na tela a única raiz; caso seja positivo, será impresso na tela as duas raízes reais pela fórmula de Bháskara: $x = \frac{-b \pm \sqrt{\Delta}}{2a}$. Na figura 1, está o código fonte à esquerda e alguns resultados obtidos variando o valor dos coeficientes (executados com `./tarefa-1.exe`).



```
program tarefa1
implicit none

real :: a,b,c,root1,root2,delta

print*, "Coeficientes a, b e c:"
read(*,*) a,b,c

delta = b * b - 4 * a * c

if (delta .gt. 0) then
    print*, "Duas raízes reais: "
    root1 = (-b + sqrt(delta))/(2 * a)
    root2 = (-b - sqrt(delta))/(2 * a)
    print*, root1, root2

else if (delta .eq. 0) then
    print*, "Uma raiz real: "
    root1 = (-b)/(2 * a)
    print*, root1

else
    print*, "Nenhuma raiz real."
end if

end program tarefa1
```

```
a12624850@ametista10:~/Documentos/p
Coeficientes a, b e c:
1 -5 6
Duas raízes reais:
3.00000000 2.00000000
a12624850@ametista10:~/Documentos/p
Coeficientes a, b e c:
2. 2e0 2
Nenhuma raiz real.
a12624850@ametista10:~/Documentos/p
Coeficientes a, b e c:
1 4 4
Uma raiz real:
-2.00000000
a12624850@ametista10:~/Documentos/p
Coeficientes a, b e c:
1 0 0
Uma raiz real:
-0.00000000
a12624850@ametista10:~/Documentos/p
Coeficientes a, b e c:
1 0 1
Nenhuma raiz real.
a12624850@ametista10:~/Documentos/p
```

Figura 1: Código fonte da tarefa 1 (à esquerda) e alguns resultados obtidos (à direita).

2 Tarefa 2

Problema: dado dois vetores, determinar a área do triângulo formado por estes.

A área do triângulo entre dois vetores em $\mathbb{R}^3$ é igual a metade da área do paralelogramo formado por estes. Este, por sua vez é igual ao módulo do produto vetorial entre os dois vetores. Portanto, foi calculado o módulo do produto vetorial entre dois vetores de entrada (de três dimensões) e o resultado foi dividido por dois. A figura 2 mostra o código fonte à esquerda e testes feitos à direita.

<pre> program tarefa2 implicit none real :: x1,y1,z1,x2,y2,z2 ! vetores de entrada real :: xr,yr,zr ! vetor resultante print*, "Coordenadas do vetor 1:" read(*,*) x1,y1,z1 print*, "Coordenadas do vetor 2:" read(*,*) x2,y2,z2 ! produto vetorial xr = y1 * z2 - y2 * z1 yr = x2 * z1 - x1 * z2 zr = x1 * y2 - x2 * y1 ! módulo do vetor é dobro da área do triângulo print*, "Área do triângulo: " print*, sqrt((xr**2) + (yr**2) + (zr**2))/2 end program tarefa2 </pre>	<pre> a12624850@ametista10:~/Docume Coordenadas do vetor 1: 1 0 0 Coordenadas do vetor 2: 0 1 0 Área do triângulo: 0.500000000 a12624850@ametista10:~/Docume Coordenadas do vetor 1: 1 -1 0 Coordenadas do vetor 2: -1 1 0 Área do triângulo: 0.000000000 a12624850@ametista10:~/Docume Coordenadas do vetor 1: 1 0 0 Coordenadas do vetor 2: 0.5 0.5 0.5 Área do triângulo: 0.353553385 a12624850@ametista10:~/Docume </pre>
---	---

Figura 2: Código fonte da tarefa 2 (à esquerda) e alguns resultados obtidos (à direita).

3 Tarefa 3

Problema: ler N números de um arquivo, imprimir o valor de N e salvar os M menores números em um arquivo de saída.

Em primeiro lugar, o arquivo de entrada foi baixado através do comando "wget" do BASH. Depois, foi descoberto o número de linhas do arquivo através do comando "wc -l" (embora esse número também tenha sido descoberto pelo programa, só foi feita essa etapa para saber o quanto de espaço seria necessário ceder para o vetor que armazenará todos esses números e não gastar memória desnecessariamente. Alternativamente, podia também ter alocado espaço para o vetor dinamicamente). O arquivo é lido e armazenado em um vetor, enquanto o número de linhas lido é impresso no terminal.

Em seguida, para imprimir os m menores números do arquivo, é preciso ordenar a lista de números primeiro. Para tal, foi empregado o algoritmo de ordenação "Heap sort". Esse algoritmo foi escolhido por alguns motivos: é rápido (complexidade de tempo $O(n \log n)$ em média, e o pior caso não difere muito em tempo do caso médio) ; é relativamente simples de implementar comparado a outros algoritmos de ordenação mais otimizados ; não precisa de recursão (Fortran 77 não admite recursão, e vários algoritmos de ordenação eficientes usam recursão, como o "quick sort" e "merge sort", entre outros. Embora esses algoritmos admitam versão iterativa, são mais complicados de implementar) ; não precisa de memória auxiliar (é um algoritmo que ordena "in place").

"Heap sort" ([1] - Knuth, D: p.144-155.) funciona da seguinte maneira: os

dados devem ser representados como uma árvore binária, com o maior elemento no topo e os menores em baixo (conhecido como um "max heap", ou apenas "heap". Mais precisamente, um vetor será um "heap" se para todo elemento de índice j , o elemento de índice $\lfloor j/2 \rfloor$ é maior ou igual a este). Isso é feito começando na metade do vetor e indo em direção ao começo, sempre impondo que o elemento no dobro do índice desse é menor (ou seja, isso verifica que o valor dos elementos no topo da árvore são sempre maiores que os elementos debaixo). Se isso for o caso, então naquela sub-árvore está tudo certo e o programa procede pra "baixo" na árvore e repete esse processo (i.e., o código dobra o índice de novo). No código, essa etapa se encerra e o vetor vira um "heap" quando x vira um (o começo do vetor foi atingido). Feito isso, é só colocar o elemento do topo (maior) no último lugar do vetor e escolher entre os dois elementos de baixo quem vai ocupar o lugar do topo. Depois repetir esse processo para ver quem vai ocupar o lugar daquele elemento que subiu pro topo e assim por diante (vai "descendo" a árvore até que se obtenha outro "heap" com menos elementos a serem considerados). O vetor então se ordena "de trás pra frente", com os maiores elementos, depois os menores. No código, a ordenação se encerra quando y vira 1 (nesse caso, o valor de "temporary" é o último elemento que sobrou do "heap" e este é colocado no começo da lista. Depois, a função retorna nada e encerra com o vetor original ordenado).

Depois de ordenar os números, são colocados os m menores números em um arquivo de saída. A figura 3 mostra o código fonte dessa tarefa sem a subrotina "Heap sort", a figura 4 mostra a subrotina "heap sort" e a figura 5 mostra o arquivo de saída para m igual a 20.

N.B.: Uma possibilidade para otimizar essa tarefa talvez poderia ser feito lendo o valor de m antes de ordenar o vetor e depois usando "heap sort" com a estrutura "min heap" ao invés de "max heap" (os menores elementos ocupam o topo da árvore binária). A possível vantagem disso seria que o vetor se ordena em ordem crescente, então assim que m elementos estão ordenados, o algoritmo para de ordenar e já escreve pro arquivo de saída esses m elementos. Essa medida não foi implementada porque não tinha certeza se teria muito ganho de tempo em média, porque teria que verificar a cada troca de elemento se completaram-se os m elementos ordenados (o que tornaria o código mais lento), e porque na própria ordenação, o algoritmo já é muito rápido (a parte lenta se encontra na criação do "heap"). Além disso, se m for próximo de N , com certeza seria mais lento que a versão implementada. Uma alternativa seria implementar essa maneira de resolução e comparar com o código já feito para ver até que valor de m o algoritmo deixa de ser eficiente. Feito isso, as duas maneiras poderiam ser implementados no código e somente um poderia ser usado dependendo do valor de m .

```

implicit none
real*8 :: arr(500000)
integer :: i , length, flag, m

open(unit=1,file="tarefa-3-entrada-1.in")
flag = 0
i = 1
do while (flag .eq. 0)
    read(1,*,iostat=flag) arr(i)
    i = i + 1
end do
close(1)

! i acaba sendo incrementado um a mais que o verdadeiro tamanho
! a cada iteração do laço e mais uma vez quando chega no EOF.
length = i - 2
print*, "Total de números: ",length
print*, "valor de m: "
read(*,*) m

call Heap_Sort(arr,length)

open(unit=2,file="tarefa-3-saida-1.dat")
write(2,*) m
do i = 1,m
    write(2,*) arr(i)
end do
close(2)

contains

subroutine Heap_Sort(array,length)

```

Figura 3: Código fonte da tarefa 3 sem a subrotina "Heap sort".

```

subroutine Heap_Sort(array,length)
  integer , intent(in) :: length
  real*8 , intent(in out) :: array(length)
  integer :: i, j, x, y
  real*8 :: temporary

  x = length / 2
  y = length
1  continue
  if (x .gt. 1) then
    temporary = array(x)
    x = x - 1
  else
    temporary = array(y)
    array(y) = array(1)
    y = y - 1
    if (y .eq. 1) then
      array(1) = temporary
      return
    end if
  end if
  i = x
  j = 2*x
  do while (j .le. y)
    if (j .lt. y) then
      if (array(j) .lt. array(j+1)) then
        j = j + 1
      end if
    end if

    if (temporary .lt. array(j)) then
      array(i) = array(j)
      i = j
      j = 2*j
    else
      j = y + 1
    end if
  end do
  array(i) = temporary
  goto 1
end subroutine Heap_Sort
end

```

Figura 4: Subrotina "heap sort" incluída no código da tarefa 3.

```

a12624850@ametista10:~/Documentos/projeto-1/tarefa-3$ cat ta
20
6.6380016505718231E-006
6.6407956182956696E-006
1.9087921828031540E-005
2.2660940885543823E-005
2.3539178073406219E-005
3.1753443181514740E-005
4.1628722101449966E-005
4.3858308345079422E-005
4.7957990318536758E-005
4.8963818699121475E-005
6.1507336795330048E-005
6.1914790421724319E-005
7.6727475970983505E-005
7.8147277235984802E-005
7.9223420470952988E-005
8.7061896920204163E-005
9.1947149485349655E-005
9.6141826361417770E-005
1.0469835251569748E-004
1.0815449059009552E-004
a12624850@ametista10:~/Documentos/projeto-1/tarefa-3$

```

Figura 5: Conteúdo do arquivo de saída quando m é igual a 20 (mostrado com comando "cat").

4 Tarefa 4

Problema: Calcular quantos e quais números primos são menores que N e salvar o resultado em um arquivo de saída.

O método empregado para essa tarefa foi usar o crivo de Eratóstenes. Essa técnica de achar números primos consiste em criar um vetor de valores booleanos (ou "logical" em fortran) de tamanho N e inicialmente marcar todo valor como verdadeiro. Os índices representarão os próprios números e o valor booleano representa se o índice é primo ou não. Começando pelo índice 2 (primeiro número primo), é marcado como falso cada múltiplo deste até N . Em seguida, acha-se o próximo número que está verdadeiro e repete essa estratégia. No final, todos os índices dos valores verdadeiros serão os números primos entre 1 e N .

Algumas otimizações feitas são: o algoritmo para se o valor verdadeiro a ser verificado é maior que $\sqrt{N}$, pois se for maior que isso, o próximo múltiplo será maior que N ; o algoritmo declara um vetor de tamanho muito grande para evitar alocar dinamicamente valores; somente é inicializado como verdadeiro os índices ímpares de 3 até N (se for par, o número não é primo - com exceção do 2, então nem é gasto tempo para inicializá-los).

Na figura 6, está o código fonte, que lê o valor de N , executa o crivo de Eratóstenes e salva os números primos em um arquivo de saída. Nas figuras 7 e 8 estão o começo e final do arquivo de saída quando $N = 10000$.

```

implicit none

integer :: n, i, j, primes
logical :: numbers(1000000000)

print*, "Números primos até: "
read(*,*) n

if (n .lt. 0) then
    print*, "Inválido."
else if (n .le. 1) then
    print*, "Não há primos antes de ", n
else
    ! crivo de eratosthenes
    numbers(2) = .true.
    numbers(3:n:2) = .true.
    do i = 3, int(sqrt(real(n))), 2
        if (numbers(i) .eqv. .true.) then
            do j = i * i, n, i
                numbers(j) = .false.
            end do
        end if
    end do
    primes = 1
    do i = 3, n, 2
        if (numbers(i) .eqv. .true.) then
            primes = primes + 1
        end if
    end do

    open(unit=1, file="tarefa-4-saida-1.dat")
    write(1,*) "total: ", primes
    write(1,*) 2
    do i = 3, n, 2
        if (numbers(i) .eqv. .true.) then
            write(1,*) i
        end if
    end do
    close(1)
end if
end

```

Figura 6: Código fonte da tarefa 4 (Crivo de Eratóstenes).


```
total: 1229
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
53
59
61
67
71
73
79
83
89
97
101
103
```

Figura 7: Primeiras linhas do arquivo de saída da tarefa 4 (primeiros 10000 números primos).

```
9743
9749
9767
9769
9781
9787
9791
9803
9811
9817
9829
9833
9839
9851
9857
9859
9871
9883
9887
9901
9907
9923
9929
9931
9941
9949
9967
9973
```

Figura 8: Últimas linhas do arquivo de saída da tarefa 4 (primeiros 10000 números primos).

Vale a pena citar que existem algoritmos mais otimizados que o crivo de Eratóstenes empregado para essa tarefa, como o crivo de Atkins e o crivo de Eratóstenes com fatoração de roda, por exemplo (cuja grande vantagem é ma-

nejar melhor a memória). Não foram empregados esses algoritmos porque são complicados para implementar e o crivo de Eratóstenes tradicional já funciona razoavelmente bem para os códigos testes (rodam em menos de um segundo para achar números primos até cerca de 10 milhões).

5 Tarefa 5

Problema: Calcular o valor do logaritmo natural usando a série de Taylor:

$$\ln x = - \sum_{N=1}^{\infty} \frac{(1-x)^N}{N}$$

Para ambas partes dessa questão, o raciocínio foi o mesmo: somar cada termo da série a uma variável e parar quando a precisão for maior que o especificado. Vale a pena notar que essa soma da série de Taylor diverge para valores de x maiores que 2 e não é definido para valores negativos de x .

Algumas otimizações feitas foram: ao invés de calcular a N -ésima potência de cada termo, foi somente multiplicado o termo seguinte por $(1-x)$ e o laço foi resumido; foi calculado o valor do logaritmo natural da função intrínseca de fortran apenas uma vez antes de iniciar o laço para evitar cálculos desnecessários quando é comparada a precisão; foi usado precisão relativa entre a soma da série com a função intrínseca ao invés da precisão absoluta, pois foi verificado experimentalmente, na parte de precisão dupla, que é possível obter um algarismo significativo a mais de precisão.

5.1 Precisão simples

Na figura 9, está o código fonte da tarefa 5a ("total sum" é o valor calculado pela série de Taylor e "simple result" é o valor da função intrínseca do Fortran). Na figura 10 estão alguns resultados obtidos.

```

implicit none

real :: x, cte, n, total_sum, accuracy, simple_result, next_term

print*, "Digite valor de x em ln(x):"
read(*,*) x
accuracy = 1e-5
cte = 1e0 - x      ! para evitar calcular isso toda vez no laço
n = 2e0
total_sum = -cte
simple_result = log(x) ! idem
next_term = cte

do while (abs((total_sum - simple_result)/ simple_result) .gt.
* accuracy)

    next_term = next_term * cte
    total_sum = total_sum - next_term / n
    n = n + 1e0

end do
print*, "Resultado da soma: ", total_sum
print*, "Função intrínseca: ", simple_result
end

```

Figura 9: Código fonte da tarefa 5a (logaritmo natural com precisão simples usando a série de Taylor).

```

a12624850@ametista10:~/Documentos/projeto-1/tarefa-5a$ .
Digite valor de x em ln(x):
1.33
Resultado da soma:  0.285180181
Função intrínseca:  0.285178989
a12624850@ametista10:~/Documentos/projeto-1/tarefa-5a$ .
Digite valor de x em ln(x):
0.45
Resultado da soma:  -0.798503101
Função intrínseca:  -0.798507750
a12624850@ametista10:~/Documentos/projeto-1/tarefa-5a$ .
Digite valor de x em ln(x):
0.998
Resultado da soma:  -2.00197427E-03
Função intrínseca:  -2.00197683E-03

```

Figura 10: Alguns resultados obtidos do código fonte da tarefa 5a.

5.2 Precisão dupla

Na figura 11, está o código fonte da tarefa 5b (a única diferença do código fonte da tarefa 5a é que a variável "simple result" vira "double result" por ter precisão dupla, assim como as demais variáveis). Na figura 12, estão os mesmos resultados da figura 10, porém com precisão dupla.

```

implicit none

real*8 :: x, cte, n, total_sum, accuracy
real*8 :: double_result, next_term, positives, negatives

print*, "Digite valor de x em ln(x):"
read(*,*) x
accuracy = 1e-15
cte = 1e0 - x      ! para evitar calcular isso toda vez no laço
n = 2e0
total_sum = -cte
double_result = dlog(x) ! idem
next_term = cte

! erro relativo dos resultados mais precisos
do while (abs((total_sum - double_result) / double_result) .gt.
& accuracy)

    next_term = next_term * cte
    total_sum = total_sum - next_term / n
    n = n + 1e0

end do
print*, "Resultado da soma: ", total_sum
print*, "Função intrínseca: ", double_result
end

```

Figura 11: Código fonte da tarefa 5b (logaritmo natural com precisão dupla usando a série de Taylor).

```

a12624850@ametista10:~/Documentos/projeto-1/tarefa-5b$ ./
Digite valor de x em ln(x):
1.33
Resultado da soma: 0.28517894223366219
Função intrínseca: 0.28517894223366247
a12624850@ametista10:~/Documentos/projeto-1/tarefa-5b$ ./
Digite valor de x em ln(x):
0.45
Resultado da soma: -0.79850769621777096
Função intrínseca: -0.79850769621777162
a12624850@ametista10:~/Documentos/projeto-1/tarefa-5b$ ./
Digite valor de x em ln(x):
0.998
Resultado da soma: -2.0020026706730788E-003
Função intrínseca: -2.0020026706730793E-003

```

Figura 12: Alguns resultados obtidos do código fonte da tarefa 5b.

6 Tarefa 6

Problema: calcular as N raízes complexas da equação $(z - 2)^N = 3$.

Geometricamente, esse problema busca determinar o vetor no plano com-

plexo cuja multiplicação consigo mesmo N vezes resulta em três (esse vetor é $(z - 2)$, então basta somar dois a este que obtém o resultado da equação). A seguir, será suposto que um vetor no plano complexo pode ser representado da seguinte maneira: $v = v_0 e^{i\theta}$, onde V_0 é o seu módulo e θ é o seu ângulo com o eixo real (ou argumento).

Logicamente, o módulo desse vetor tem que ser $3^{1/N}$, porque N multiplicações sucessivas desse vetor consigo mesmo multiplica o seu módulo N vezes (o módulo de 3 é 3).

O ângulo, por sua vez, tem que coincidir com o argumento do vetor 3, i.e., tem que ser $\theta = 2\pi \times x$, onde x é um número inteiro qualquer. Como a multiplicação de um vetor consigo mesmo rotaciona o vetor no sentido anti-horário em um valor de θ , o vetor em questão tem que ter ângulo igual a $\theta = \frac{2\pi \times x}{N}$, porque N sucessivas rotações terão que coincidir com o ângulo de 3 (isso equivale a somar θ , N vezes ao ângulo do vetor).

O algoritmo empregado para essa tarefa usa a fórmula de Euler para facilitar as contas: $V_0 e^{i\theta} = V_0 (\cos(\theta) + i \sin(\theta))$, onde $\theta = \frac{2\pi n}{N}$ com $n \in [1, N]$. Cada solução da equação em questão é calculado dessa maneira e depois 2 é somado a sua parte real para levar em conta o menos dois na equação original com z .

A figura 13 mostra o código fonte da tarefa 6 e a figura 14 mostra alguns resultados variando o valor de N .

```
implicit none

integer :: n, x
complex :: z
real :: two_pi_over_n, A

print*, "N = "
read(*,*) n

! para evitar calcular toda vez no laço
two_pi_over_n = 2e0*acos(-1e0) / real(n)
A = 3e0**(1e0/real(n))
print*, "Soluções no plano complexo:"
do x = 1, n
  z = cmplx((A*cos(two_pi_over_n*x)+2e0), (A*sin(two_pi_over_n*x)))
  write(*, '(F0.5,sp,F0.5,"J ")') z
end do
end
```

Figura 13: Código fonte da tarefa 6 (achar raízes complexas da equação $(z - 2)^N = 3$).

```

a12624850@ametista10:~/Document
N =
1
  Soluções no plano complexo:
5.00000+0.00000j
a12624850@ametista10:~/Document
N =
4
  Soluções no plano complexo:
2.00000+1.31607j
.68393-.00000j
2.00000-1.31607j
3.31607+.00000j
a12624850@ametista10:~/Document
N =
15
  Soluções no plano complexo:
2.98297+.43764j
2.71998+.79962j
2.33250+1.02333j
1.88753+1.07010j
1.46201+.93183j
1.12951+.63245j
.94752+.22371j
.94752-.22371j
1.12951-.63245j
1.46201-.93183j
1.88753-1.07010j
2.33250-1.02333j
2.71998-.79962j
2.98297-.43764j
3.07599+.00000j

```

Figura 14: Alguns resultados obtidos do código fonte da tarefa 6.

Uma otimização feita foi usar uma variável para armazenar o valor de $\frac{2\pi}{N}$, para evitar fazer essa conta toda vez no argumento das funções trigonométricas. Vale a pena observar que o código poderia ser mais otimizado se levasse em conta a simetria do problema, i.e., usar o fato de que alguns valores de seno e cosseno se repetem para ângulos entre 0 e 2π (como $\sin(\pi/4) = \sin(3\pi/4)$, ou $\cos(\pi/6) = \cos(-\pi/3)$, por exemplo). Porém, não foi feita essa otimização porque tornaria o código desnecessariamente mais complicado de entender para ter um ganho mínimo de tempo.

7 Tarefa 7

Problema: Calcular o volume de uma esfera em d dimensões usando o método Monte Carlo e a fórmula $V_d = \frac{\pi^{d/2}}{\Gamma(1+d/2)} R^d$.

A resolução dessa tarefa consiste na utilização do método de Monte Carlo: números aleatórios permeiam uma região a ser considerada e a razão de pontos dentro de uma região específica com o total de pontos é proporcional ao volume da região específica.

Inicialmente é dado um "seed" à função "rand" do Fortran (experimentalmente foi observado que o "seed"19 proporciona resultados mais precisos). Depois foi calculado a distância de cada ponto aleatório à origem, e foi verificado se este é menor ou igual ao raio da esfera. Isso foi feito elevando cada componente do ponto ao quadrado, somando tudo a uma variável e tomando a raiz quadrada de tudo. Se a distância desse ponto for menor ou igual ao raio da esfera, o número de pontos dentro aumenta em um. No final, o volume total é a razão de pontos dentro com o total dos pontos, vezes o volume do cubo associado à esfera (a aresta do cubo é o diâmetro da esfera, e o volume do cubo é a aresta elevado ao valor da dimensão em que o cubo se encontra).

Em seguida, foi comparado o volume obtido do método de Monte Carlo com a fórmula do volume da esfera em d dimensões. Para implementar tal função, basta escrever uma versão simples e mais restrita da função gamma. Como as dimensões serão apenas números naturais (incluindo zero), a função gamma receberá argumentos que serão, ou números naturais, ou números naturais somados com meio. No primeiro caso, a função gamma será apenas o fatorial do número menos um: $\Gamma(x) = (x-1)! \iff x \in \mathbb{N}$. No segundo caso, a função gamma será dada por: $\Gamma(x) = \sqrt{\pi} \prod_{n=1}^{x-0.5} \frac{2n-1}{2} \iff x \in \{w+0.5 | w \in \mathbb{N}\}$.

Uma otimização feita foi reescrever a fórmula do volume da esfera da seguinte maneira: $\frac{\pi^{d/2}}{\Gamma(1+d/2)} R^d = \frac{(\sqrt{\pi} \times R)^d}{\Gamma(1+d/2)}$, o que facilita as contas para dimensões altas (não tem que ficar chamando a função intrínseca de exponenciação duas vezes).

Na figura 15, está o código fonte da tarefa 7. Na figura 16, estão alguns resultados aleatórios. Na figura 17, estão os resultados obtidos fixando a dimensão e raio da esfera, mas variando o número de pontos aleatórios no método de Monte Carlo (a acurácia do método Monte Carlo fica cada vez melhor).

```

integer :: i, j, n
real :: d, r, v1, v2, x, total, inside, distance
! d real, pois tem precisao um pouco melhor
! total e inside real pois senao a divisao fica truncada no final

print*, "Dimensão e raio da esfera: "
read(*,*) d, r
print*, "Número de pontos aleatórios para Monte Carlo: "
read(*,*) total
x = rand(19)
inside = 0.
do i = 1, int(total)
distance = 0.
do j = 1, int(d)
distance = distance + (r * rand())**2
end do
distance = sqrt(distance)
if (distance .le. r) then
inside = inside + 1.
end if
end do
v2 = (inside / total) * ((2. * r)**d)
v1 = volume_esfera(d,r)
print*, "Volume da esfera (pela fórmula): ", v1
print*, "Volume da esfera (Monte Carlo): ", v2

contains

function volume_esfera(d,r)
implicit none
real, intent(in) :: r, d
real, parameter :: SQRTPI = sqrt(acos(-1.))
real :: n, volume_esfera, fatorial
integer :: i

if (mod(int(d),2) .eq. 0) then
! Gamma == fatorial
fatorial = 1.
do i=1,int(d/2)
fatorial = fatorial * i
end do
volume_esfera = ((SQRTPI * r)**d) / fatorial
return
else
n = 1.
volume_esfera = ((SQRTPI * r)**d) / SQRTPI
do i = 1, int(d), 2
volume_esfera = volume_esfera * 2. / n
n = n + 2.
end do
return
end if
end function volume_esfera
end

```

Figura 15: Código fonte da tarefa 7 (método Monte Carlo para achar volume de esfera em d dimensões e implementação da fórmula do volume da esfera em d dimensões).


```

a12624850@ametista10:~/Documentos/projeto-1/tarefa-7$ ./
Dimensão e raio da esfera:
0 1
Número de pontos aleatórios para Monte Carlo:
100
Volume da esfera (pela fórmula):    1.00000000
Volume da esfera (Monte Carlo):    1.00000000
a12624850@ametista10:~/Documentos/projeto-1/tarefa-7$ ./
Dimensão e raio da esfera:
3 2
Número de pontos aleatórios para Monte Carlo:
10000
Volume da esfera (pela fórmula):    33.5103226
Volume da esfera (Monte Carlo):    33.9584007
a12624850@ametista10:~/Documentos/projeto-1/tarefa-7$ ./
Dimensão e raio da esfera:
10 7
Número de pontos aleatórios para Monte Carlo:
1000000
Volume da esfera (pela fórmula):    720358272.
Volume da esfera (Monte Carlo):    720244160.

```

Figura 16: Alguns resultados obtidos do código fonte da tarefa 7.

```

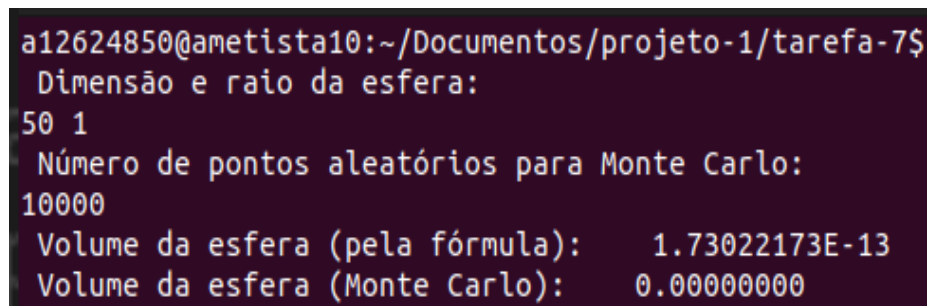
a12624850@ametista10:~/Documentos/projeto-1/tarefa-
Dimensão e raio da esfera:
3 1
Número de pontos aleatórios para Monte Carlo:
10
Volume da esfera (pela fórmula):    4.18879032
Volume da esfera (Monte Carlo):    5.59999990
a12624850@ametista10:~/Documentos/projeto-1/tarefa-
Dimensão e raio da esfera:
3 1
Número de pontos aleatórios para Monte Carlo:
100
Volume da esfera (pela fórmula):    4.18879032
Volume da esfera (Monte Carlo):    4.07999992
a12624850@ametista10:~/Documentos/projeto-1/tarefa-
Dimensão e raio da esfera:
3 1
Número de pontos aleatórios para Monte Carlo:
1000
Volume da esfera (pela fórmula):    4.18879032
Volume da esfera (Monte Carlo):    4.39200020
a12624850@ametista10:~/Documentos/projeto-1/tarefa-
Dimensão e raio da esfera:
3 1
Número de pontos aleatórios para Monte Carlo:
1000000
Volume da esfera (pela fórmula):    4.18879032
Volume da esfera (Monte Carlo):    4.19432783

```

Figura 17: Resultados obtidos da tarefa 7 fixando o raio e dimensão, mas variando o número de pontos aleatórios.

Uma observação feita nessa tarefa foi que o método de Monte Carlo não é tão preciso quanto a fórmula do volume da esfera. Eu acredito que essa discrepância se deve, em geral, à formulação dos números aleatórios, i.e., os números gerados são pseudo-aleatórios, então são levemente enviesados, o que proporciona mais pontos estarem dentro ou fora da esfera de maneira sistemática.

Outra observação feita é que para dimensões altas, o método de Monte Carlo fornece valores de volume iguais a zero, mesmo usando bastante pontos. Isso ocorre porque em altas dimensões, o volume da esfera tende ao zero, o que dificulta os pontos a estarem dentro da esfera. Consequentemente, se nenhum ponto atingir o interior da esfera, a razão de pontos dentro para total de pontos fica igual a zero e o volume final se anula. Esse comportamento será tratado em mais detalhes na tarefa 8. Na figura 18, está o valor nulo do volume do método de Monte Carlo para $d = 50$.



```
a12624850@ametista10:~/Documentos/projeto-1/tarefa-7$
  Dimensão e raio da esfera:
50 1
  Número de pontos aleatórios para Monte Carlo:
10000
  Volume da esfera (pela fórmula):      1.73022173E-13
  Volume da esfera (Monte Carlo):      0.00000000
```

Figura 18: Resultado obtido da tarefa 7 quando o número de dimensões é alta (Monte Carlo falha porque o volume da esfera é muito pequeno).

8 Tarefa 8

Problema: Calcular o volume de uma esfera de raio r em d dimensões e plotar o resultado usando `xmgrace` como função da dimensão.

8.1 Fórmula

A fórmula de volume de uma esfera em d dimensões é dado pela fórmula: $V_d = \frac{\pi^{d/2}}{\Gamma(1+d/2)} R^d$. Sua implementação já foi feita na tarefa 7 e o algoritmo foi apenas copiado para esta parte (por isso que a fórmula foi implementado como uma função: para facilitar sua utilização posterior).

A novidade é que foi calculado o volume da esfera $d + 1$ vezes (uma para cada dimensão, de 0 a d) e estes valores foram armazenados em um arquivo de saída. A figura 19 mostra o código fonte da tarefa 8a, e a figura 20 mostra o arquivo de saída quando $d = 20$.

```

implicit none
integer :: i
real :: d, r, v

print*, "Dimensão e raio da esfera: "
read(*,*) d, r
open(unit=1,file="tarefa-8a-saida-1.dat")
write(1,*) "Dimensão ", "      Volume da esfera"
do i=0,int(d)
    v = volume_esfera(real(i),r)
    write(1,*) i, v
end do
close(1)
contains

function volume_esfera(d,r)
implicit none
real, intent(in) :: r, d
real, parameter :: SQRTPI = sqrt(acos(-1.))
real :: n, volume_esfera, fatorial
integer :: i

if (mod(int(d),2) .eq. 0) then
    ! Gamma == fatorial
    fatorial = 1.
    do i=1,int(d/2)
        fatorial = fatorial * i
    end do
    volume_esfera = ((SQRTPI * r)**d) / fatorial
    return
else
    n = 1.
    volume_esfera = ((SQRTPI * r)**d) / SQRTPI
    do i = 1, int(d), 2
        volume_esfera = volume_esfera * 2. / n
        n = n + 2.
    end do
    return
end if
end function volume_esfera
end

```

Figura 19: Código fonte da tarefa 8a (algoritmo para calcular o volume de uma esfera de d dimensões).

Dimensão	Volume da esfera
0	1.0000000000
1	2.0000000000
2	3.14159274
3	4.18879032
4	4.93480301
5	5.26378965
6	5.16771364
7	4.72476673
8	4.05871296
9	3.29850984
10	2.55016470
11	1.88410449
12	1.33526325
13	0.910629034
14	0.599264741
15	0.381443471
16	0.235330731
17	0.140981182
18	8.21459293E-02
19	4.66216281E-02
20	2.58069057E-02

Figura 20: Resultado obtido da tarefa 8a dentro do arquivo de saída quando $d = 20$.

8.2 Visualização

Primeiramente, foi empregado o algoritmo da tarefa 8a (subseção "Fórmula") para escrever em um arquivo de saída (em cada coluna): as dimensões, os volumes das esferas de raios 0.9, 1.0 e 1.1 em cada dimensão. Ou seja, a primeira coluna são as dimensões de 0 a 25; a segunda coluna é o volume da esfera de raio 0.9 para as dimensões de 0 a 25 ; a terceira coluna é o volume da esfera de raio 1.0 para as dimensões de 0 a 25 etc.

Uma vez que esse arquivo foi criado, foi usada o seguinte comando no terminal: "xmgrace -nxy tarefa-8b-saida-1.dat", o que cria um gráfico .agr plotando cada coluna como função da primeira (Nesse caso, os volumes são plotados em função da dimensão). Na figura 21, está o código fonte da tarefa 8b. Na figura 22, está o gráfico dos volumes de três esferas de raios diferentes em função da dimensão em que se encontram.

```

implicit none
integer :: d
real :: r1, r2, r3, v

r1 = 0.9
r2 = 1.
r3 = 1.1
open(unit=1,file="tarefa-8b-saida-1.dat")
do d= 0, 25
    write(1,*) d, volume_esfera(real(d),r1),
& volume_esfera(real(d),r2), volume_esfera(real(d),r3)
end do
close(1)
contains

function volume_esfera(d,r)
implicit none
real, intent(in) :: r, d
real, parameter :: SQRTPI = sqrt(acos(-1.))
real :: n, volume_esfera, fatorial
integer :: i

if (mod(int(d),2) .eq. 0) then
    ! Gamma == fatorial
    fatorial = 1.
    do i=1,int(d/2)
        fatorial = fatorial * i
    end do
    volume_esfera = ((SQRTPI * r)**d) / fatorial
    return
else
    n = 1.
    volume_esfera = ((SQRTPI * r)**d) / SQRTPI
    do i = 1, int(d), 2
        volume_esfera = volume_esfera * 2. / n
        n = n + 2.
    end do
    return
end if
end function volume_esfera
end

```

Figura 21: Código fonte da tarefa 8b (escreve para um arquivo de saída o valor da dimensão e dos respectivos volumes de três esferas de raios 0.9 , 1.0 e 1.1).

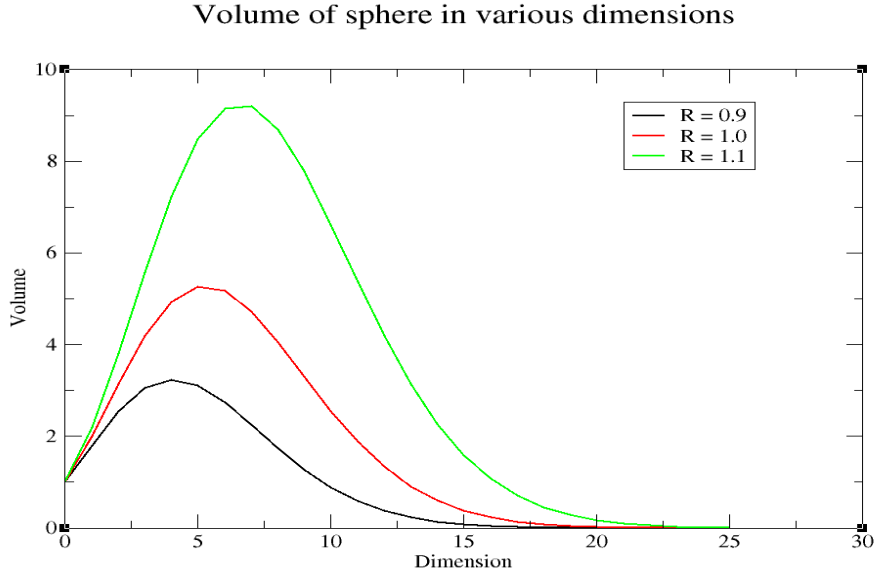


Figura 22: Volumes de três esferas de raios 0.9 , 1.0 e 1.1 como função da dimensão.

9 Tarefa 9

Problema: Comparar o volume de uma esfera com a de um cubo quando o número de dimensões tende ao infinito (fixando o raio da esfera e as arestas do cubo). Em seguida, calcular a ordem típica do número de Avogadro em um mundo d-dimensional.

9.1 Comparação de volumes: esfera e cubo

Um cubo de aresta de tamanho $1m$ terá volume constante e igual a $1m^d$ independentemente da dimensão. Uma esfera de raio $1m$ terá volume dado pela fórmula: $V_d = \frac{\pi^{d/2}}{\Gamma(1+d/2)}$. Pela figura 22, é verificado experimentalmente que o volume de uma esfera decresce com o aumento da dimensão, logo a razão entre o volume de um cubo e de uma esfera tende ao infinito à medida que a dimensão aumenta.

Matematicamente, a razão do volume de um cubo e de uma esfera em dimensão d é:

$$\frac{1}{\left(\frac{\pi^{d/2}}{\Gamma(1+d/2)}\right)} = \frac{\Gamma(1+d/2)}{\pi^{d/2}}$$

Como a dimensão é um número natural, terá dois casos a serem considerados: a função gamma para valores inteiros positivos e a função gamma para valores inteiros positivos somados a meio.

No primeiro caso, a função gamma vira o fatorial do antecessor do argu-

mento, e a razão fica (d é par):

$$\frac{(d/2)!}{\pi^{d/2}} = \frac{(x)!}{\pi^x}, \text{ onde } x \in \mathbb{N}$$

Agora considere a sequência onde o enésimo termo é dado por $a_n = \frac{n!}{\pi^n}$. Pelo teste da razão, é possível mostrar que essa sequência diverge:

$$\lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = \lim_{n \rightarrow \infty} \frac{\frac{(n+1)!}{\pi^{(n+1)}}}{\frac{n!}{\pi^n}} = \lim_{n \rightarrow \infty} \frac{n+1}{\pi} = \infty$$

Ou seja, isso significa que o fatorial cresce mais rápido que o exponencial. Isso mostra que a razão do volume de um cubo com a de uma esfera tende ao infinito à medida que as dimensões tendem ao infinito.

Para o segundo caso, um argumento análogo pode ser empregado. A razão entre os volumes de um cubo e esfera fica:

$$\frac{\Gamma(1+d/2)}{\pi^{d/2}} = \frac{\sqrt{\pi} \prod_{n=1}^{(1+d/2)-0.5} \frac{2n-1}{2}}{\pi^{d/2}} = \frac{\prod_{n=1}^{(d/2+1/2)} \frac{2n-1}{2}}{\pi^{(d/2-1/2)}}$$

Como d é ímpar, seja $x = d/2 - 1/2$, onde $x \in \mathbb{N}$. Então a expressão fica:

$$\frac{\prod_{n=1}^{x+1} \frac{2n-1}{2}}{\pi^x} = \frac{\prod_{n=1}^{x+1} (2n-1)}{2(2 \times \pi)^x} = \frac{(2x-1)!!}{2(2 \times \pi)^x}$$

Onde, por definição, seja $x, k \in \mathbb{N}$, então $x!! = \prod_{k=0}^{\lceil x/2 \rceil - 1} (x-2k)$. Por exemplo, $5!! = 1 \times 3 \times 5$.

Novamente considere a sequência onde o enésimo termo seja $a_n = \frac{(2n-1)!!}{2(2 \times \pi)^n}$. Pelo teste da razão, demonstrar-se-á que essa série diverge:

$$\lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = \lim_{n \rightarrow \infty} \frac{\frac{(2n+1)!!}{2(2 \times \pi)^{n+1}}}{\frac{(2n-1)!!}{2(2 \times \pi)^n}} = \lim_{n \rightarrow \infty} \frac{2n+1}{2\pi} = \infty$$

Com isso, fica provado que a razão entre os volumes de um cubo e esfera tende ao infinito à medida que o número de dimensões tende ao infinito. ■

Além disso, para uma dada dimensão par, o volume do cubo será $\frac{(d/2)!}{\pi^{d/2}}$ vezes maior que o volume de uma esfera. Enquanto para uma dimensão ímpar, o volume do cubo será $\frac{(2x-1)!!}{2(2 \times \pi)^x}$ vezes maior que o volume de uma esfera (onde $x = d/2 - 1/2$).

9.2 Número de Avogadro

Sejam os volumes de uma proteína, átomo e objeto macroscópico: $1\mu m^d$, $1\text{\AA}^d$ e $1mm^d$. O número de Avogadro é a quantidade de átomos em uma molécula, então será dado por:

$$\frac{1\mu m^d}{1\text{\AA}^d} = \left(\frac{10^{-6}}{10^{-10}} \right)^d = 10^{4d}$$

Porém, existe um erro nessa conta, porque quando $d = 3$, o número de Avogadro deve ser $\approx 6 \times 10^{23}$ e não 10^{12} . Para coibir esse erro sistemático, será multiplicado o valor do número de Avogadro em dimensão d por $\frac{6 \times 10^{23}}{10^{12}} = 6 \times 10^{11}$.

Portanto, o número de Avogadro em dimensão d é aproximadamente dado por: $6 \times 10^{4d+11}$.

10 Bibliografia

[1] - Knuth, Donald. "The Art Of Computer Programming. Vol. 3, Sorting and Searching", ed. 2. Stanford: Addison-Wesley, 2011. ISBN: 978-0-201-89685-5. p. 144-155.